main.py

- Define seed airfoil
- Runs the full project workflow
    - Calls optimizer, scoring, aero solver, etc.
- Saves results from optimizer using log.py
- Makes plots using plotting.py

config.py

- Stores constants and settings

  Track_weights = {
  “Z_mode” : 0.53
  “X_mode” : 0.047
  }
  Design bounds = {
  "alpha": (-5.0, 15.0),
  "thickness": (0.06, 0.18),
  "thickness_loc": (0.20, 0.50),
  "camber": (0.00, 0.08),
  "camber_loc": (0.20, 0.60),
  }
    - Different bounds for phase 1, 2, 3

  Penalty settings
    - solver settings (number of max iterations, tolerance, etc)

design_variables.py

- Define the design variables of the airfoil (create a dataclass)
    - Different for phase 1 and 2 vs 3

  @dataclass
  class AirfoilDesign:
  alpha: float
  thickness: float
  thickness_loc: float
  camber: float
  camber_loc: float

geometry.py

- Takes design variables as input
- Convert airfoil design_variables into airfoil coordinates
- Output x and y coordinates (160 points maybe?)
    - Likely just choose which specific points on the airfoil can change based on the design variable

scoring.py

- Converts aerodynamic outputs into a score.

- Inputs: cl, cd, mode, etc
- Output: total score

aero_interface.py
- Connects Python code to MSES/Pymead.
- Inputs: airfoil geometry
- Outputs: aerodynamics variables (cl, cd, etc.)

optimizer.py
- Calls aero_interface.py and scoring.py repeatedly to optimize geometry

log.py
- Saves cl, cd, score, etc from each iteration into an ongoing txt/csv file

plotting.py
- Creates plots for final report
    - Score vs iteration
    - Aerodynamics variables vs iterations
- Plot geometric representation for seed airfoil vs final optimized design